Neural Networks and Deep Learning - Programming Assignments

Chiara Zara

December 19, 2025

Contents

1 Task 1: Supervised learning basics **2**

2 Task 2: Double Descent **7**

3 Task 3: Transfer Learning **12**

4 Task 4: Autoencoders **17**

5 Task 5: Generative Models **20**

1 Task 1: Supervised learning basics

Model Architectures

Two different neural network architectures were considered in order to compare their behavior and performance on image classification.

The first model is a Multi-Layer Perceptron (MLP). Each input image of size 28×28 pixels is flattened into a vector of 784 features. The network consists of two fully connected hidden layers with 400 neurons each, followed by an output layer with 10 neurons corresponding to the digit classes. ReLU activation functions are applied after each hidden layer to introduce non-linearity and to allow the model to learn non-trivial decision boundaries. This architecture was chosen as baseline, since it represents a standard supervised classifier that does not explicitly exploit the spatial structure of images and therefore provides a useful reference point.

The second model is a Convolutional Neural Network (CNN). It processes the images using two convolutional layers with 3×3 kernels, with 32 and 64 feature maps respectively. Each convolution is followed by a ReLU activation and a 2×2 max pooling operation, which reduces spatial resolution and introduces translation invariance. The extracted features are then passed through a fully connected layer with 128 neurons and finally to a 10-dimensional output layer. This architecture was selected because convolutional networks are well suited for image data and are expected to outperform fully connected models on vision tasks.

For both models, the output layer produces a 10-dimensional vector of class scores. During training, these scores are converted into class probabilities through the softmax operation implicitly included in the cross-entropy loss. The cross-entropy loss was chosen because it encourages correct probabilistic predictions and provides a well-behaved objective for gradient-based optimization.

Data Splitting

The MNIST dataset was divided into three disjoint subsets to ensure a reliable evaluation procedure. First, the official MNIST dataset already provides a separation between training data (60,000 images) and test data (10,000 images). The test set was kept completely separate and was used only for the final evaluation of the selected models.

The original training set was further split into a training set of 50,000 samples and a validation set of 10,000 samples. The training set was used to update the model parameters during optimization, while the validation set was used to monitor generalization performance and to compare different hyperparameter choices.

Training and Hyperparameter Selection

Both models were trained using mini-batch optimization with a batch size of 128, which represents a compromise between variance reduction in the gradient estimates and training efficiency, without introducing excessive memory or computational cost. For the baseline configuration, the Adam optimizer with learning rate 3×10^{-4} was used. This choice was motivated by the fact that Adam combines adaptive learning rates with momentum-like

behavior, which generally leads to faster and more stable convergence compared to plain stochastic gradient descent. Each model was trained for 5 epochs, which was sufficient to achieve good optimization on the training set without signs of overfitting.

The optimization process was explicitly monitored on the training data by plotting the training loss as a function of the number of iterations. These plots provide a direct way to assess whether the objective function is being minimized effectively and whether the learning dynamics are stable.

To highlight the importance of hyperparameter selection, two intentionally poor configurations were also tested. In the first case (Poor A), an extremely large learning rate (LR = 0.9) was used together with Stochastic Gradient Descent. This choice was made to deliberately cause the parameter updates to overshoot the minimum of the loss function. Since standard SGD uses a fixed step size and, in this setup, does not include momentum, it is unable to compensate for such an aggressive learning rate. As a result, the optimization becomes numerically unstable, leading to divergence of the loss and training accuracy close to random guessing.

In the second case (Poor B), the learning rate was kept at a reasonable value, but the number of training epochs was reduced to one. In this scenario, the optimization process is interrupted too early, preventing the model from sufficiently minimizing the training loss and resulting in underfitting.

For the MLP, the effect of these hyperparameter choices is illustrated in Figure 1. With the good configuration, the training loss decreases smoothly and consistently across iterations, indicating stable gradient-based optimization. In contrast, Poor A shows clear divergence of the loss due to unstable updates, while Poor B exhibits only a limited decrease of the loss, reflecting insufficient training.

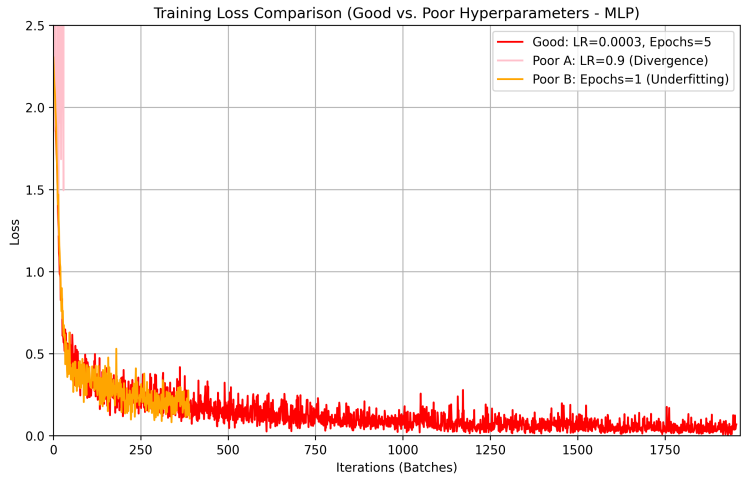

Figure 1: Training loss comparison for the MLP using good and poor hyperparameter choices. The good configuration converges smoothly, while Poor A diverges and Poor B underfits.

A similar behavior can be observed for the CNN, as shown in Figure 2. When using the selected hyperparameters, the training loss decreases rapidly and remains stable, showing that the optimization procedure is effective also for a deeper and more structured model.

As for the MLP, Poor A leads to unstable optimization due to the excessively large learning rate, while Poor B limits the model's ability to minimize the loss because of the reduced number of training epochs.

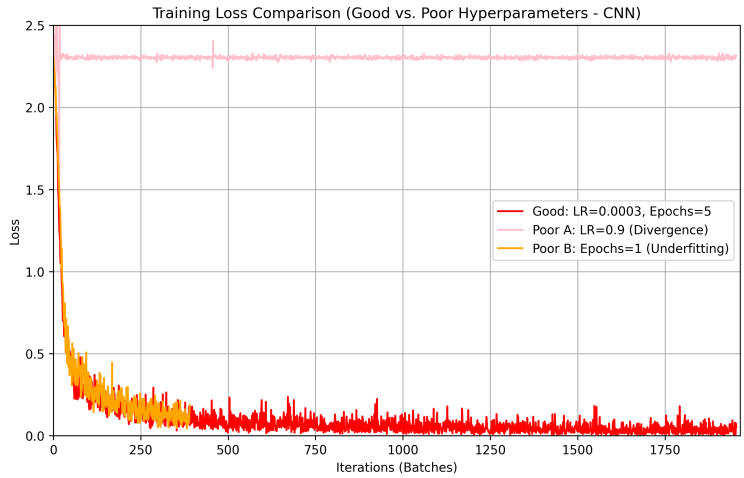

Figure 2: Training loss comparison for the CNN with good and poor hyperparameter choices. Stable convergence is observed only for the well-tuned configuration.

The training loss curves clearly demonstrate how appropriate choices of optimizer, learning rate, and training duration lead to effective optimization on the training data, while poor choices result in divergence or insufficient learning.

For completeness, the training accuracy is also reported in Figure 3 and Figure 4. The accuracy curves are consistent with the loss behavior and provide an intuitive confirmation of the optimization dynamics observed in the loss plots.

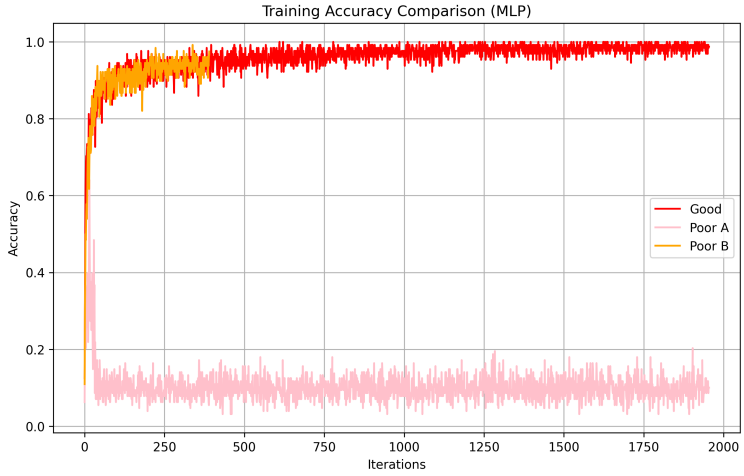

Figure 3: Training accuracy comparison for the MLP.

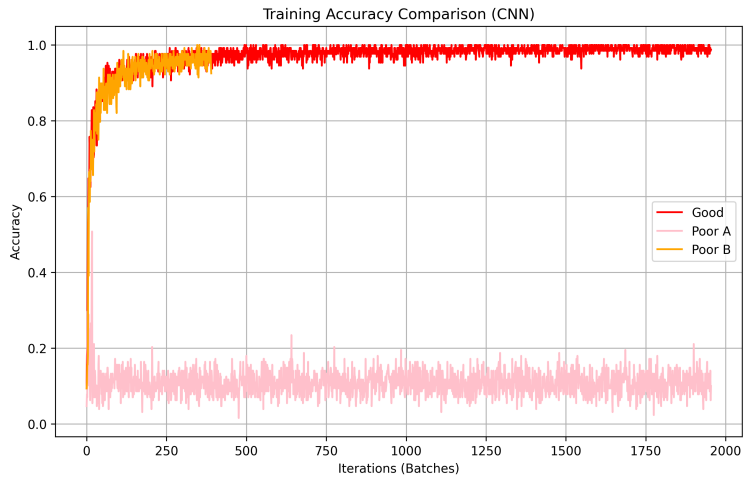

Figure 4: Training accuracy comparison for the CNN.

Model Selection and Test Results

Model selection was based on validation performance. For each architecture, the hyperparameter configuration that achieved the lowest validation loss and highest validation accuracy was selected. In practice, the validation set was used to compare different optimization settings and to ensure that improvements on the training set translated into better generalization, rather than overfitting.

After selecting the best hyperparameters using the validation data, a final evaluation was performed on the test set, which had not been used during training or model selection. The MLP achieved a test accuracy of approximately 97.8%, while the CNN reached about 98.6%. These results confirm that both models generalize well to unseen data, with the CNN providing the best overall performance, as expected for image classification tasks.

A direct comparison of the training loss for the two selected models is shown in Figure 5.

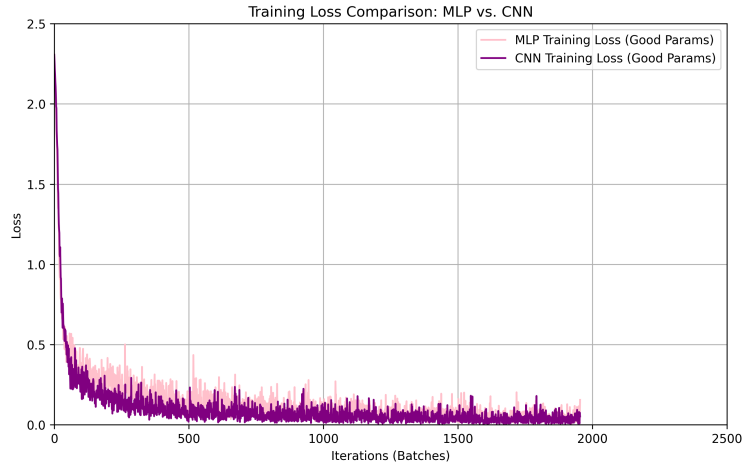

Figure 5: Training loss comparison between the MLP and the CNN using the selected hyperparameters.

Finally, qualitative examples of predictions on the test set are reported below. These examples show that the models correctly classify most digits, providing additional qualitative evidence of successful supervised learning.

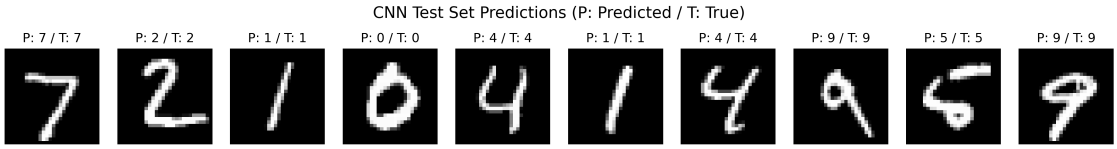

Figure 6: Example predictions of the CNN on the MNIST test set (Predicted / True labels).

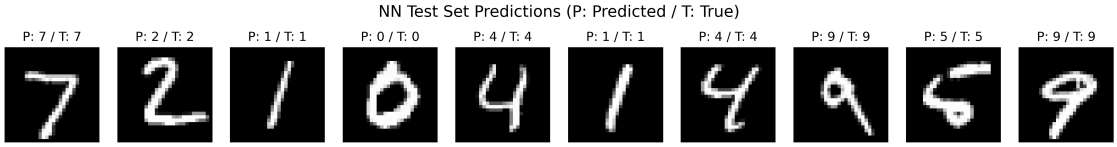

Figure 7: Example predictions of the MLP on the MNIST test set (Predicted / True labels).

2 Task 2: Double Descent

The goal of this task is to study the double descent phenomenon on the MNIST dataset. Rather than aiming for an exact reproduction of the curves shown in the reference paper, the focus was placed on designing experiments that explicitly reflect the three main aspects discussed. In particular, attention was given to how double descent emerges from the interaction between model capacity, the amount of information provided by the training data, and the optimization dynamics that determine how the model fits the data.

For this reason, two complementary experimental setups were considered. In the first approach, a standard classification setting was used, relying on stochastic gradient descent and cross-entropy loss. In the second approach, the training procedure was modified in order to reduce implicit regularization effects and better isolate the role of model capacity.

Stochastic Gradient Descent and Cross-Entropy Loss

For the first set of experiments, a one-hidden-layer Multi-Layer Perceptron (MLP) was used, where the width of the hidden layer directly controls the number of trainable parameters. The training set size was fixed to $N = 1000$ samples, randomly selected from the MNIST training data. Model complexity was varied by increasing the hidden layer width, and the total number of parameters was used as a quantitative measure of model capacity.

This setup directly addresses the first key aspect discussed in the paper: variation in model capacity while keeping the training data fixed. The chosen training set size is small enough for increasing model capacity to eventually lead to interpolation, while still being large enough to avoid trivial memorization.

For each model width, the network was trained using stochastic gradient descent with a fixed learning rate of 0.01. The number of epochs was set to 45, which was sufficient for the largest models to reach very low or zero training error, while keeping the overall training time manageable across the full range of model sizes. A mini-batch size of 128 was used in order to balance stability of the gradient updates and training efficiency, while allowing all models, including the largest ones, to be trained under the same conditions without excessive computational cost. No explicit regularization techniques such as weight decay or early stopping were applied, so that the observed behavior could be attributed primarily to the interaction between model size, data size, and optimization dynamics.

Training error and test error were recorded after training. Figure 8 shows the training and test error as functions of model complexity on a logarithmic scale.

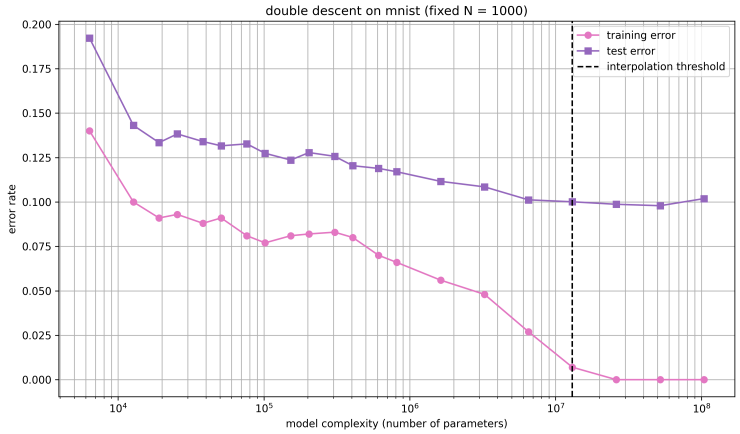

Figure 8: Training and test error as a function of model complexity for a fixed training set size.

The numerical results show a gradual decrease of training error as model capacity increases. For small models, such as width 8 (6,370 parameters), the training error remains relatively high at 0.14, with a corresponding test error of 0.19, indicating a clearly underparameterized regime. As the width increases, both training and test error decrease steadily. For example, at width 512 (approximately 4.1×10^5 parameters), the training error drops to 0.08 and the test error to 0.12.

The interpolation regime is reached only for very large models. In particular, the training error becomes effectively zero at width 32,768, corresponding to approximately 2.6×10^7 parameters. Beyond this point, training error remains zero, while test error stabilizes around 0.10. The minimum test error of 0.0979 is achieved at width 65,536, after which a slight increase to 0.1019 is observed for the largest model. This suggests that, even though interpolation is achieved, extremely large model capacity does not necessarily lead to further improvements in generalization.

Unlike the classical double descent curve, a pronounced peak in test error before interpolation is not clearly visible. Instead, the test error decreases smoothly across model sizes and remains low even beyond the interpolation threshold. This behavior is consistent with the presence of implicit regularization induced by stochastic gradient descent and cross-entropy loss, which biases the optimization toward solutions that generalize well even in the overparameterized regime. In addition, the structured nature of the MNIST dataset further mitigates sharp generalization degradation.

To further analyze the behavior around interpolation, additional visualizations were considered. Figure 9 highlights the training error and the location of the interpolation threshold, while Figure 10 shows the generalization gap, defined as the difference between test error and training error.

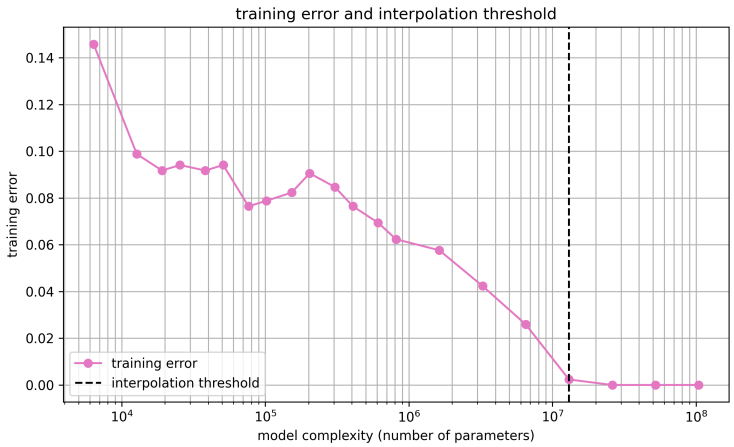

Figure 9: Training error and interpolation threshold across model complexity.

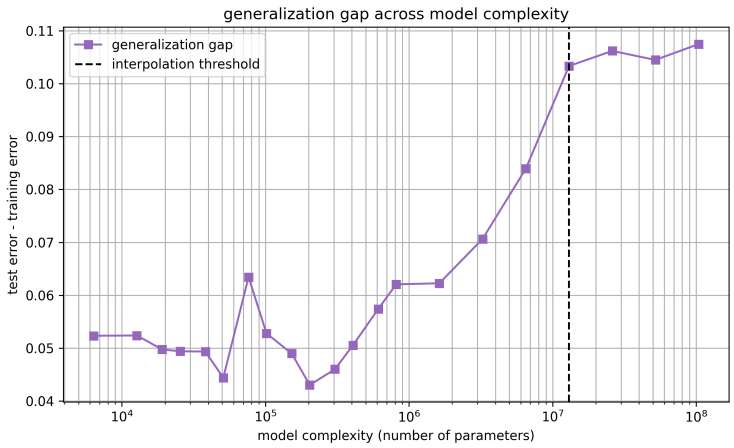

Figure 10: Generalization gap as a function of model complexity.

These results show that interpolation is clearly achieved, but the increase in test error before interpolation is limited. This suggests that the effective model capacity is smaller than the nominal number of parameters, due to implicit regularization induced by the optimization process.

Full-Batch Training with Mean Squared Error

Motivated by the smooth behavior observed in the baseline experiment, a second set of experiments was conducted to reduce the amount of implicit regularization introduced by the learning procedure. The goal of this experiment was to more clearly expose the interaction between model capacity and generalization.

Several changes were introduced. The cross-entropy loss was replaced by a mean squared error loss applied to one-hot encoded targets. Full-batch gradient descent was used instead of mini-batch training, momentum was removed, and training was performed for a larger

number of epochs. These choices significantly reduce stochasticity and remove classification-specific biases from the optimization process.

From the perspective of the three key aspects of double descent, this setup keeps the training data fixed and continues to vary model capacity, while substantially altering the optimization dynamics. As a result, the model becomes less constrained by implicit regularization, and its behavior becomes more sensitive to increases in capacity.

Figure 11 shows the training and test error obtained with this modified setup.

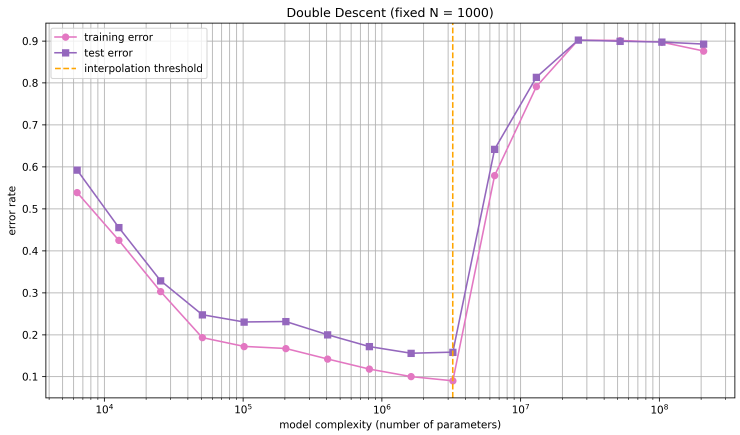

Figure 11: Double descent experiment using mean squared error and full-batch gradient descent.

In this setting, the behavior across model complexity changes substantially. While increasing model capacity initially improves both training and test performance, very large models exhibit unstable behavior and poor performance even on the training set. For example, at width 4,096 (approximately 3.3×10^6 parameters), the training error reaches 0.09 and the test error 0.16. However, beyond this point, both training and test error increase sharply. At width 16,384, the training error rises to 0.79 and the test error to 0.81, approaching random guessing. This indicates that, in the absence of implicit regularization, increasing capacity alone does not guarantee successful fitting or generalization.

This experiment highlights that the observed generalization behavior is not only a property of the architecture, but of the entire learning setup. The loss function and optimization method strongly influence how model capacity translates into effective fitting of the data.

Conclusion

Across the two experimental setups, several characteristics of the double descent phenomenon were observed. Fixing the training set size while increasing model capacity leads to an interpolation regime where training error approaches zero. However, the behavior of the test error strongly depends on the optimization dynamics and the loss function.

Under standard classification training with stochastic gradient descent and cross-entropy loss, implicit regularization smooths the transition around interpolation, resulting in a gradual improvement of test performance. When implicit regularization is reduced, the sensitivity

to model capacity becomes much more pronounced, and overly complex models can suffer from instability and degraded performance.

These results are consistent with the insights discussed in the paper: double descent emerges from the interaction between data, model capacity, and optimization.

3 Task 3: Transfer Learning

In this task, transfer learning was investigated using the MNIST dataset, starting from a convolutional neural network trained in Task 1. The original model was trained to classify handwritten digits using the unmodified data images and achieved high accuracy on the corresponding test set. For this reason, it was considered a suitable source model for transfer learning, as its convolutional layers are expected to have learned meaningful and reusable visual features, such as edges, strokes, and local patterns, that can potentially generalize to related tasks.

Rather than considering a single transfer scenario, two related experiments of increasing difficulty were designed. The motivation behind this choice was to first analyze a setting where the difference between the original task and the new task is relatively limited, and then move to a more challenging scenario where the input data is transformed more substantially. This progressive approach makes it possible to better understand under which conditions transfer learning provides clear benefits and when adaptation becomes necessary due to a stronger change in the data distribution.

Definition of the New Tasks

To define new tasks that differ from the original one while keeping the same digit classes, the input data was modified by applying rotations to the images. Two rotation settings were considered. In the first experiment the digits were randomly rotated by up to 30 degrees, but this lead to many digits remaining visually similar to their original appearance, and with their main structural properties preserved. So, in the second experiment, the digits were rotated by exactly 90 degrees, which significantly changes their visual appearance and makes recognition more challenging without specific adaptation.

The same training hyperparameters were used in all transfer learning experiments, both for the 30-degree and the 90-degree rotation tasks, in order to ensure a fair comparison. A batch size of 64 was selected as a balance between stable gradient estimates and sufficient variability during training, which is particularly important when working with small datasets. The learning rate was fixed to 10^{-3}, a value that allowed effective optimization with the Adam optimizer without causing unstable updates. The number of training epochs was set to 10, which was sufficient for both the scratch and transfer models to converge on the limited training data, while avoiding excessive overfitting. Finally, the number of training samples for the new task was fixed to $N = 300$,in order to simulate a few-shot learning scenario. Performance was always evaluated on the full rotated MNIST test set, so that the reported results reflect generalization rather than performance on the limited training subset.

Transfer Learning Method

The transfer learning strategy adopted in this task follows a feature-based approach. The convolutional layers of the pre-trained CNN were frozen and used as a fixed feature extractor, while only the fully connected layers were retrained on the new task. This choice is suitable for the considered setup, since it reduces the number of trainable parameters and allows the

model to reuse visual representations learned from the original dataset, which is particularly important when training data is limited.

To ensure a fair comparison, the same architecture, optimizer, learning rate, batch size, and number of training epochs were used for all models. For each new task, three solutions were evaluated:

- (a) the original pre-trained model evaluated directly on the new task without adaptation,
- (b) a model trained from scratch using only the small training set of the new task,
- (c) the transfer learning model obtained by freezing the convolutional layers and retraining the final fully connected layers.

Results on 30-Degree Rotation

Figure 12 reports the test accuracy obtained on the dataset rotated by 30 degrees. The original model without adaptation already achieves high accuracy (93.54%), indicating that the features learned on the original MNIST data generalize well to this type of transformation.

Training a model from scratch using only a limited number of rotated samples leads to lower performance (84.54%), reflecting the difficulty of learning robust representations from few examples. The transfer learning approach achieves the best result, with an accuracy of 95.69%, showing that reusing pre-trained convolutional features helps the model adapt more efficiently in this setting.

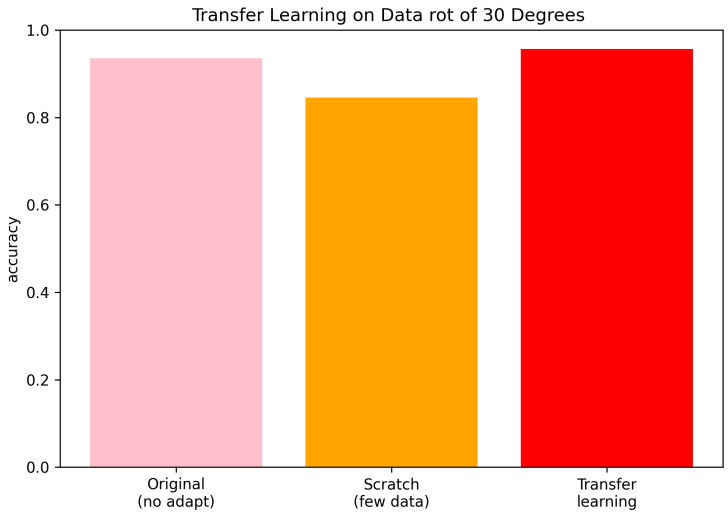

Figure 12: Test accuracy comparison for the 30-degree rotated MNIST task.

Training accuracy and training loss were monitored to better understand the learning dynamics. Figures 13 and 14 show that the transfer learning model reaches high accuracy in fewer epochs and exhibits a smoother decrease in training loss, while the scratch model

improves more slowly. This indicates that transfer learning leads to faster convergence and more stable optimization.

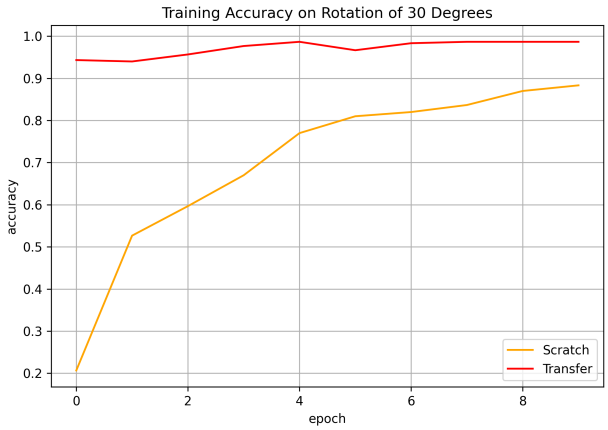

Figure 13: Training accuracy over epochs on the 30-degree rotated task.

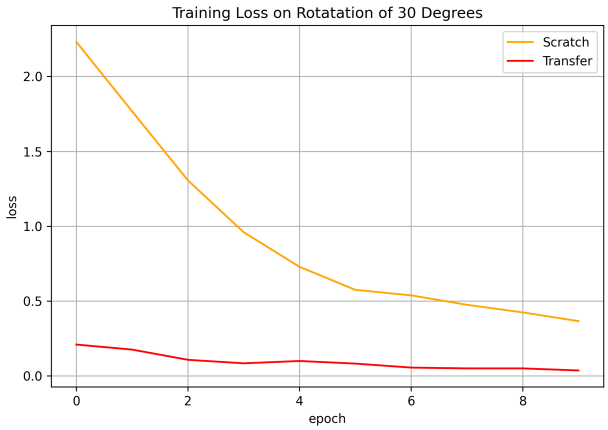

Figure 14: Training loss over epochs on the 30-degree rotated task.

The strong performance of the original model in this setting suggests that a 30-degree rotation does not strongly alter the structure of the data. For this reason, a more challenging scenario was considered by rotating the digits by 90 degrees.

Results on 90-Degree Rotation

The 90-degree rotation task is significantly more challenging. As shown in Figure 15, the original model without adaptation performs very poorly, achieving only 15.99% accuracy. This confirms that the features learned on the original dataset are not sufficient to handle such a strong transformation.

Training from scratch on the small rotated dataset improves performance to 87.25%, but the result is still limited by the small number of training samples. The transfer learning

model achieves the highest accuracy (90.38%), showing that reusing and adapting learned features provides a clear advantage when the input distribution changes substantially.

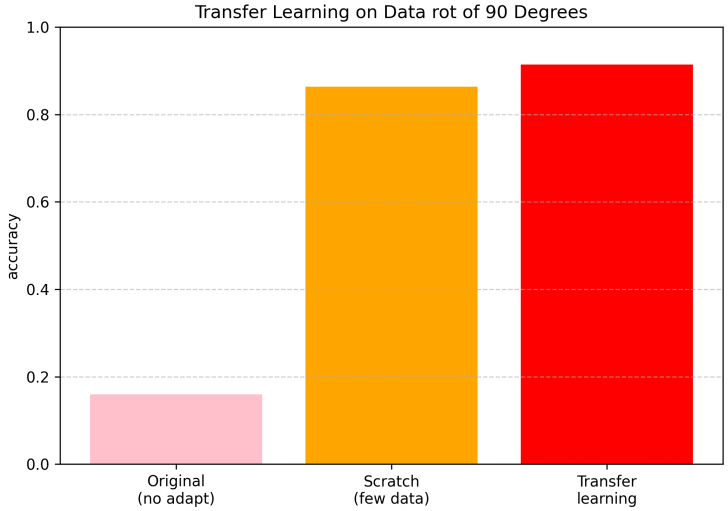

Figure 15: Test accuracy comparison for the 90-degree rotated MNIST task.

The training curves in Figures 16 and 17 highlight further differences between the two training strategies. The transfer learning model converges faster and shows a smoother reduction in training loss, while the scratch model requires more epochs to reach similar performance. This behavior indicates more stable optimization when knowledge from the original task is reused.

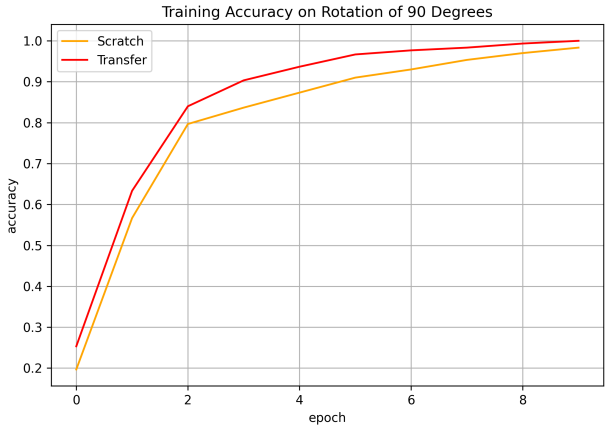

Figure 16: Training accuracy over epochs on the 90-degree rotated task.

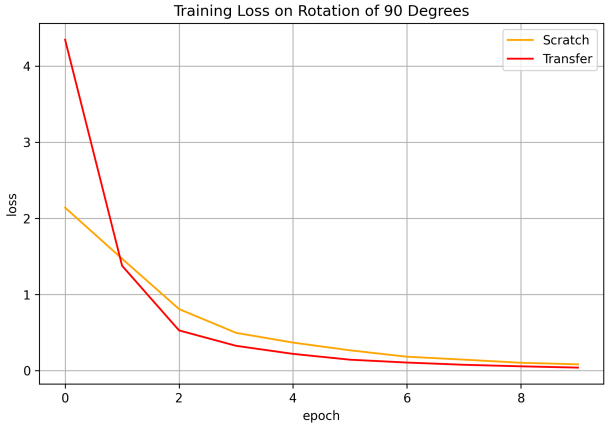

Figure 17: Training loss over epochs on the 90-degree rotated task.

Conclusion

These experiments show that transfer learning is particularly effective when training data for a new task is limited. When the transformation applied to the input data is moderate, the pre-trained model already performs well, but fine-tuning still improves convergence speed and final accuracy. When the transformation is much stronger, the original model fails without adaptation, while transfer learning clearly outperforms training from scratch.

The results demonstrate how previously learned visual representations can be reused to improve learning efficiency and stability when addressing new but related tasks under data constraints.

4 Task 4: Autoencoders

In this task, a dimension-reducing autoencoder was trained on the MNIST dataset with the objective of learning a compact two-dimensional representation of handwritten digits. Two autoencoder architectures were considered, both following the same design principles: the encoder and decoder have the same depth, and the decoder mirrors the structure of the encoder. This choice allows the comparison to focus specifically on the effect of network depth, while keeping the rest of the architecture fixed.

The input images are flattened into vectors of dimension $28 \times 28 = 784$, and a fully connected architecture was selected. This avoids introducing additional design choices, such as convolutional structures, that could obscure the interpretation of how depth alone influences the learned latent space. In both models, the bottleneck layer contains exactly two units, enforcing a strong dimensionality reduction.

The first architecture is a shallow autoencoder with a single hidden layer before the bottleneck, while the second architecture adds an extra hidden layer on both the encoder and decoder sides, increasing the expressive capacity of the network. ReLU activations are used in all hidden layers to introduce nonlinearity, while no activation function is applied at the output layer, since reconstruction quality is directly optimized using mean squared error loss on pixel values. Both autoencoders were trained using the mean squared error loss, which directly measures reconstruction quality in pixel space, since the objective is to reproduce the input rather than perform classification. Optimization was performed using the Adam optimizer with a learning rate of 10^{-3}, a value that provided stable convergence without requiring extensive tuning. Training was carried out for 40 epochs, which was sufficient for both models to reach a stable reconstruction performance, while avoiding unnecessary overtraining. A batch size of 256 was used to ensure efficient training and smooth gradient estimates on the full MNIST dataset.

After training, the encoder of each autoencoder was applied to a fixed subset of 500 randomly selected test images. Their two-dimensional latent representations were then visualized by plotting the encoded points on the plane. Coloring the points by digit label makes it possible to assess how well the autoencoder organizes the data in the compressed space and how the architectural depth affects the structure of the learned representation.

Depth 2 Autoencoder

Figure 18 shows the latent space learned by the shallow autoencoder with depth 2.

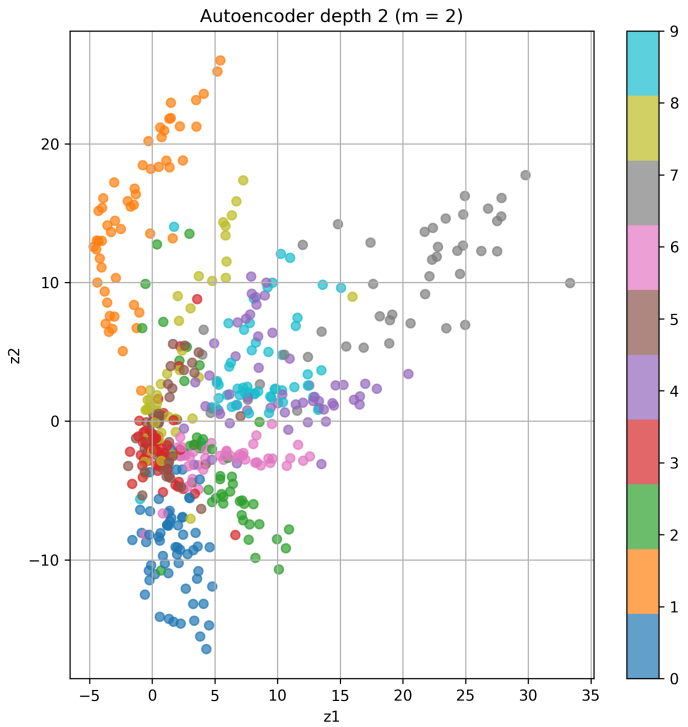

Figure 18: 2D latent representation learned by the depth 2 autoencoder.

The latent representation is clearly compressed, but the separation between different digit classes is limited. Many classes overlap in the central region of the latent space, and several clusters appear mixed or elongated rather than compact. In particular, digits with similar structural features such as 3, 5, and 8 show significant overlap near the origin, while classes 4 and 9 also appear partially mixed. This behavior can be attributed to the limited expressive power of the shallow encoder, which must map a high-dimensional and complex data distribution into two dimensions using only a small number of nonlinear transformations.

Despite these limitations, some degree of structure is still visible. Digits that share similar visual characteristics tend to be mapped closer together, indicating that the encoder has learned basic shape-related features even under strong dimensional constraints. However, finer class-specific details are often lost, leading to significant overlap between classes.

Depth 3 Autoencoder

Figure 19 shows the latent space learned by the deeper autoencoder with depth 3.

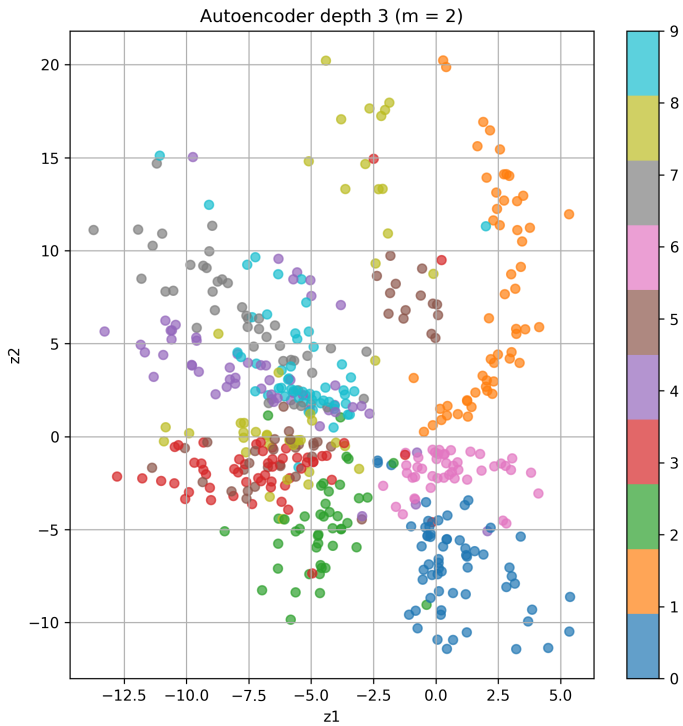

Figure 19: 2D latent representation learned by the depth 3 autoencoder.

Compared to the shallow model, the latent space learned by the deeper autoencoder is more structured. Many digit classes form tighter and more coherent clusters, and the global organization of the space appears more regular. The additional hidden layer in the encoder allows the model to apply a richer sequence of nonlinear transformations before reaching the bottleneck, which improves its ability to capture relevant factors of variation in the data.

This increased depth enables the network to better approximate the low-dimensional structure underlying the MNIST digits and to compress it into two dimensions in a more informative way. Some overlap between classes is still present, given the low dimensionality of the latent space, but the improvement over the shallow architecture is clearly visible.

Conclusion

These experiments demonstrate how architectural depth influences the quality of learned low-dimensional representations in autoencoders. When the bottleneck dimension is fixed to $m = 2$, increasing the depth of the encoder leads to a better organization of the latent space and improved separation between digit classes.

The results suggest that deeper networks are better suited to represent complex nonlinear mappings, even when the final representation is severely constrained. At the same time, the remaining overlap between classes highlights the inherent trade-off between dimensionality reduction and information preservation. Overall, this task illustrates how autoencoders can be used for dimensionality reduction and exploratory data analysis, and how architectural choices directly affect the geometry of the learned latent space.

5 Task 5: Generative Models

In this task, a generative model based on a Generative Adversarial Network was trained on the MNIST dataset. The objective was to generate new handwritten digit images and the emphasis was placed on implementing a correct GAN setup and analyzing the training dynamics and generated outputs.

The implemented solution follows a DCGAN-style architecture adapted to the MNIST dataset. In this context, DCGAN-style refers to the use of convolutional networks in both the generator and the discriminator, together with design choices that are used to stabilize GAN training. In particular, the generator is built using transposed convolutional layers to progressively transform a low-dimensional latent vector into an image, while the discriminator uses standard convolutional layers to analyze images at different spatial resolutions.

Batch normalization is applied in most layers of both networks to improve training stability, and ReLU and LeakyReLU activations are used in the generator and discriminator respectively. These choices help the networks learn meaningful spatial features and reduce issues such as vanishing gradients during training.

The architecture was adapted to MNIST by using a single image channel and a relatively small network depth. The model was intentionally kept compact in order to limit computational cost and reduce training instability, while still preserving the key characteristics of a DCGAN-style design.

Model Architecture and Training Setup

The generator receives as input a latent vector $z \in \mathbb{R}^{100}$ sampled from a standard normal distribution and produces a 28×28 grayscale image. It is composed of a sequence of transposed convolutional layers with batch normalization and ReLU activations. A final Tanh activation is applied to match the normalization of the MNIST images, which are scaled to the range $[-1, 1]$.

The discriminator is a convolutional neural network that takes an image as input and outputs a single probability indicating whether the image is real or generated. It consists of convolutional layers with LeakyReLU activations and batch normalization, followed by a Sigmoid output layer. This structure allows the discriminator to capture spatial patterns while remaining relatively lightweight.

Both networks were trained using the Adam optimizer with learning rate 2×10^{-4} and $\beta_1 = 0.5$. These values were chosen to balance stability and convergence speed. Binary cross-entropy loss was used for both the generator and the discriminator, reflecting the binary classification nature of the discriminator's task.

A batch size of 128 was used during training, this value represents a compromise between stable gradient estimates and computational efficiency, while still allowing sufficient stochasticity to support adversarial learning. The model was trained for 200 epochs on the MNIST training set. A relatively large number of epochs was required because training was progressing slow and exhibiting high variance; while running for fewer epochs was resulting in poorly structured samples. Training beyond this point would significantly increase computation time without guaranteeing consistent improvements.

During training, generated samples were saved at selected epochs to visually inspect how the generator evolves over time, and the losses of both networks were recorded to analyze training dynamics.

Evolution of Generated Samples

Figure 29 shows generated samples at different stages of training. At the very beginning (epoch 1), the generator produces images that resemble random noise, with no recognizable digit structure. This is expected, since the generator has not yet learned how to map the latent space to meaningful images.

As training progresses (epochs 25, 50), digit-like patterns start to appear. Basic strokes and curves become visible, but many samples are still distorted or incomplete. This indicates that the generator is beginning to better capture visual features, while still lacking fine-grained structure.

At later stages (epochs 75 to 200), the generated images become more recognizable. Several digits clearly resemble valid MNIST samples, and common shapes such as loops and straight lines are better defined. However, the quality remains uneven, with some digits appearing ambiguous or poorly formed. This suggests that the generator has learned important visual patterns but has not fully captured the full variability of the data distribution. It can also be observed that some digit shapes appear clearer at intermediate epochs than at later stages. For example, certain digits that are well formed at earlier epochs may become distorted or partially open at later epochs (such as for the zero well defined in epoch 50 and 75, but not completely closed in later stages). This behavior reflects the adversarial nature of GAN training, where improvements are not monotonic and the generator continuously changes its strategy in response to the discriminator.

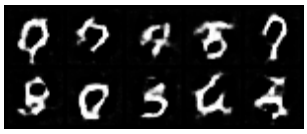

Figure 20: Epoch 1

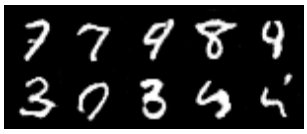

Figure 21: Epoch 25

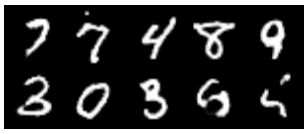

Figure 22: Epoch 50

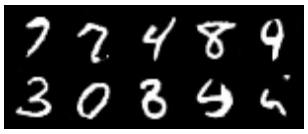

Figure 23: Epoch 75

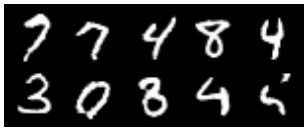

Figure 24: Epoch 100

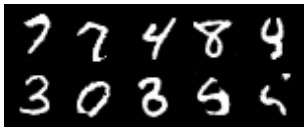

Figure 25: Epoch 125

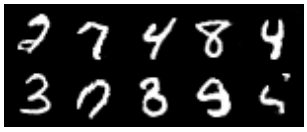

Figure 26: Epoch 150

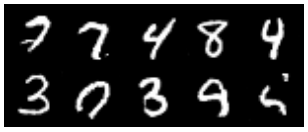

Figure 27: Epoch 175

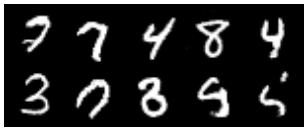

Figure 28: Epoch 200

Figure 29: Evolution of generated digits at different training epochs.

Where the last plot, Figure 28, shows a final set of generated digits after 200 epochs, illustrating the overall quality achieved by the trained generator.

The evolution of the generator and discriminator losses during training is shown in Figure 30. The loss curves exhibit strong oscillations throughout training and do not converge to stable values. This behavior reflects GAN training, where the two networks continuously adapt to each other instead of minimizing a single objective.

The discriminator loss remains relatively low on average, suggesting that it often succeeds in distinguishing real images from generated ones, and It shows large fluctuations, indicating alternating phases where the generator temporarily improves and phases where it struggles to fool the discriminator. Despite this instability, training does not collapse completely, and the generator continues to produce increasingly structured samples.

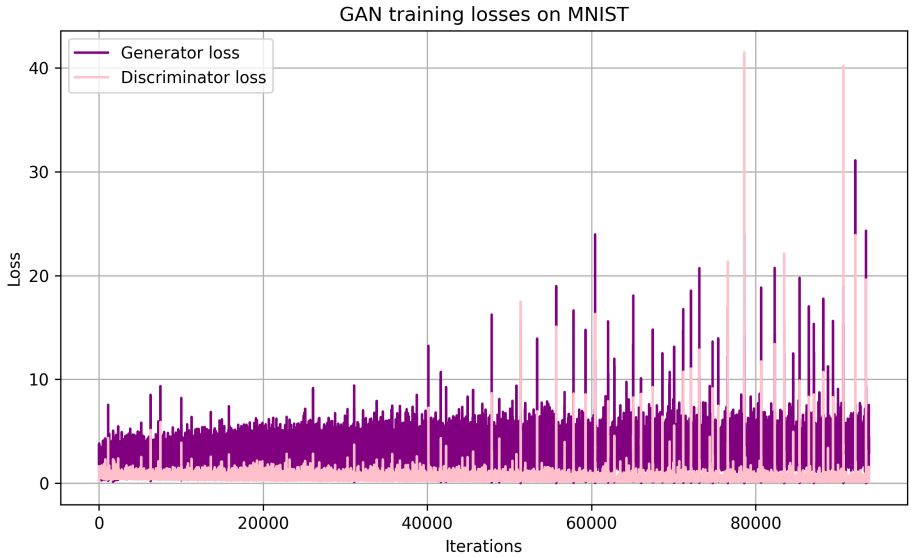

Figure 30: Generator and discriminator losses during GAN training.

Conclusion

The obtained results show that the implemented GAN is able to generate images that resemble handwritten MNIST digits. Several samples clearly exhibit recognizable digit structures, such as closed loops or straight strokes, indicating that the generator has learned meaningful visual patterns from the data.

At the same time, the quality of the generated digits is not uniform. Some digits are well formed, while others appear distorted, incomplete, or ambiguous, and in a few cases the class of the digit is difficult to identify. This can be explained by the fact that certain digit shapes are easier to model with simple convolutional features, while more complex or less frequent patterns are harder for the generator to reproduce consistently. In addition, the limited capacity of the network and the strong compression of the latent space may restrict the generator's ability to represent the full variability of the data distribution.

The training dynamics remain unstable, as reflected by the oscillatory behavior of the generator and discriminator losses. This instability indicates that the two networks contin-

uously adapt to each other rather than converging to a fixed optimum. No explicit stabilization techniques were used in this experiment, which further contributes to the observed fluctuations and to the uneven quality of the generated samples.

It is also worth noting that the model was trained for 200 epochs, which already required a significant amount of computational time. Given this constraint, no extensive hyperparameter tuning or architectural modifications were explored. Further improvements would likely require longer training, more careful parameter tuning, or additional stabilization techniques.

This experiment demonstrates that a relatively simple DCGAN-style model is sufficient to produce structured and recognizable digits, even if some samples remain imperfect.

Use of Generative AI Tools

Generative AI tools (ChatGPT) were used only as support for minor textual editing and language revision. All experimental ideas, model design choices, code implementation, and interpretation of the results are mine.